Initially: `x = y = z = 0`

p :	q :	r :	s :

`o := x; (1)`	`y := 1; (2)`	`m := y; (3)`
		`if m = 0 then`
		`  z := 1; (4)`
			`n := z; (5)`
			`l := y; (6)`
			`if n = 1 then`
			`  if l = 0 then`
			`  x := 1; (7)`

Fig. 2. Program with control flow.

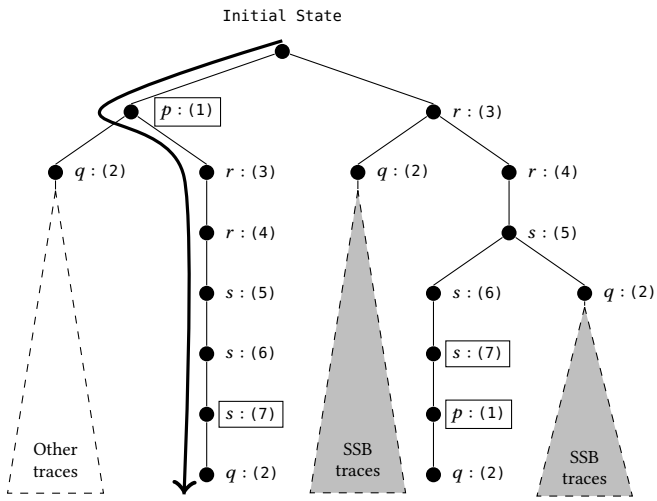

Fig. 3. Explored interleavings for the program of Fig. 2.

labeled `(5)`, `(6)`, and `(7)`. Statements that operate on local variables are assumed to be part of the previous labeled statement. For example, label `(6)` marks the read of the value of `y`, together with the assignment to `l`, and the condition check on `n`. If the value of `n` is `1`, the condition check on `l` is also part of `(6)`, which ends just before the assignment to `x` that has the label `(7)`. Similar assumptions are made for the other local statements.

Consider a DPOR algorithm that starts the exploration with p, explores the interleaving $p.r.r.s.s.s$ (marked in Fig. 3 with an arrow from top to bottom), and then detects the race between events `(1)` and `(7)`. It must then explore some interleaving in which the race is reversed, i.e., the event `(7)` occurs before the event `(1)`. Note that event `(7)` will occur only if it is preceded by the sequence `(3) - (4) - (5) - (6)` and not preceded by a step of process q. Thus, an interleaving that reverses this race must start with the sequence $r.r.s.s$. Such an interleaving is shown in Fig. 3 between the two chunks labeled "SSB traces" (Sleep Set Blocked traces).

Having detected the race in $p.r.r.s.s.s$, *source-DPOR* adds r to the source set at the initial state. However, it does not "remember" that r must be followed by $r.s.s$ to reverse the race. After exploring r, it may therefore continue with q. However, after $r.q$ any exploration is doomed to encounter sleep set blocking, meaning that the exploration reaches a state in which all enabled processes are in the sleep set. To see this, note that p is in the sleep set when exploring r, and will remain